Municipalidad de Algarrobo

Informe Técnico de Auditoría de Seguridad y Arquitectura – Plataforma “Geoportal Municipal”

Documento técnico que presenta los resultados del análisis del código fuente, evaluación de arquitectura y revisión de estándares de ciberseguridad sobre los entregables del desarrollo.

Auditoría Geoportal Municipal - SECPLAC

I. Municipalidad de Algarrobo, Departamento de Informática

Marzo 2026

Índice

1. Resumen Ejecutivo 2
2. Objetivo y Metodología 3
3. Hallazgos Críticos de Seguridad y Control de Acceso 3
3.1. Control de Acceso 3
3.2. Principio de Menor Privilegio 4
3.3. Configuración Permisiva de CORS 4
3.4. Existencia de Rutas API sin uso (Shadow APIs) 4
3.5. Asignación Insegura por Defecto (Violación de “Fail-Safe Defaults”) 5
3.6. Falta de Auditoría en la Cadena de Suministro (Dependencias de Terceros) . 5
4. Resultados de Análisis Estático de Código (SAST) 6
4.1. Riesgo de Inyección SQL en el Back-end 6
4.2. Riesgo de Cross-Site Scripting (XSS) en el Front-end 6
4.3. Integridad de Dependencias (Ataque de Cadena de Suministro) 6
5. Estabilidad de Infraestructura y Gobernanza de Datos 7
5.1. Interrupciones por mal manejo del Connection Pool (Errores 500) 7
5.2. Privacidad de Datos y Seguridad Crítica en IA (Chatbot) 7
5.3. Filtración de Credenciales y Código Sensible Expuesto 7
5.4. Credenciales por Defecto y Datos Semilla Inseguros 8
5.5. Operaciones con Transaccionalidad Insuficiente en Flujos Críticos 8
5.6. No se implementa JWT y Sesiones Volátiles 9
6. Higiene del Repositorio y Deuda Técnica 9
6.1. Falta de Limpieza y Archivos Residuales 9
6.2. Enlaces Rígidos (Hardcoding de IP) en el Front-end 9
6.3. Interfaces Duplicadas (Shadow UI) 10
6.4. Bug de Sobrescritura de Datos (Pérdida de Contraseñas) 10
6.5. Manejo Deficiente de Excepciones en el Cliente (Enmascaramiento de Errores) 10
6.6. Diccionario de Rutas Incompleto y Bucles de Redirección 11
7. Inconsistencias en la Documentación Técnica 11
8. Problemas de Usabilidad en Dispositivos Móviles (Diseño Responsivo) 12
9. Alcance del Proyecto y Módulos no Solicitados 13
10. Conclusión General 13

1. Resumen Ejecutivo

El sistema evaluado presenta vulnerabilidades críticas de seguridad y deficiencias arquitectónicas que comprometen la integridad, confidencialidad y disponibilidad de la información.

Se identificaron fallas graves en:

- Control de acceso y autorización
- Manejo de credenciales y sesiones
- Construcción de consultas a base de datos
- Gobernanza de datos y exposición a servicios externos

En su estado actual, el sistema no cumple con estándares mínimos requeridos para su despliegue en un entorno de producción institucional.

Se recomienda condicionar la recepción del proyecto a la subsanación de los siguientes problemas identificados:

N°	Hallazgo / Vulnerabilidad	Prioridad de Resolución
3.1	Control de Acceso	**Bloqueante**
3.2	Principio de Menor Privilegio	**Bloqueante**
3.3	Configuración Permisiva de CORS	**Alta**
3.4	Existencia de Rutas API sin uso (Shadow APIs)	**Alta**
3.5	Asignación Insegura de Roles por Defecto	**Bloqueante**
3.6	Falta de Auditoría en Dependencias de Terceros	**Alta**
4.1	Riesgo de Inyección SQL en el Back-end	**Bloqueante**
4.2	Riesgo de Cross-Site Scripting (XSS)	**Alta**
4.3	Integridad de Dependencias (Ataque de Cadena de Suministro)	**Media**
5.1	Interrupciones por mal manejo de Conexiones (Error 500)	**Media**
5.2	Privacidad de Datos y Seguridad Crítica en IA (Chatbot)	**Alta**
5.3	Filtración de Credenciales y Código Sensible Expuesto	**Bloqueante**
5.4	Credenciales por Defecto y Datos Semilla Inseguros	**Bloqueante**
5.5	Operaciones con Transaccionalidad Insuficiente en Flujos Críticos	**Alta**
5.6	No se implementa JWT y Sesiones Volátiles	**Alta**
6.1	Falta de Limpieza y Archivos Residuales	**Media**
6.2	Enlaces Rígidos (Hardcoding de IP) en el Front-end	**Media**
6.3	Interfaces Duplicadas (Shadow UI)	**Media**
6.4	Bug de Sobrescritura de Datos (Pérdida de Contraseñas)	**Bloqueante**
6.5	Manejo Deficiente de Excepciones en el Cliente	**Media**
6.6	Diccionario de Rutas Incompleto y Bucles de Redirección	**Media**
7.0	Inconsistencias en la Documentación Técnica	**Alta**
8.0	Problemas de Usabilidad en Dispositivos Móviles	**Media/Baja**
9.0	Alcance del Proyecto y Módulos no Solicitados	**Media**

- Bloqueante: debe corregirse antes de recepción conforme.
- Alta: debe corregirse con prioridad en el corto plazo.
- Media: planificable, no impide recepción si existe compromiso/plan.

2. Objetivo y Metodología

El presente documento tiene como propósito visibilizar las deudas técnicas, fallas arquitectónicas y vulnerabilidades críticas presentes en la plataforma entregada, las cuales deben subsanarse antes de que el sistema sea autorizado para operar en un entorno de producción institucional.

- **Alcance:** La auditoría consistió en una revisión exhaustiva del código fuente (Front-end y Back-end), combinando inspección manual con análisis estático automatizado (mediante Bandit para Python y Semgrep para JavaScript/Arquitectura). Adicionalmente, se realizaron pruebas funcionales en un entorno aislado (Docker) observando el comportamiento de los datos durante el período de pruebas.
- **Limitaciones:** La revisión se restringió a la evidencia disponible en el repositorio correspondiente al enlace de pruebas entregado y no constituye una prueba de penetración dinámica (Pentesting) sobre la infraestructura final. Algunos hallazgos ambientales requieren confirmación directa en producción.

  https://lmonsalve22.github.io/ALGARROBO_BASE/frontend/index.html

  https://github.com/lmonsalve22/ALGARROBO_BASE

3. Hallazgos Críticos de Seguridad y Control de Acceso

3.1. Control de Acceso

- **Observación:** El sistema carece de validación real de permisos al acceder a las rutas. El esquema de seguridad se basa en "ocultar" los enlaces en la interfaz gráfica, pero la función de enrutamiento (**router.js**) no detiene la navegación directa ni el Back-end verifica los privilegios en la ejecución de acciones.
- **Impacto:** Se comprobó que cualquier usuario autenticado con privilegios mínimos (ej. Rol **"Transparencia", Nivel 16**) puede acceder a vistas y funciones de administración con solo modificar la URL en el navegador (ej. escribiendo **/admin_general/** en lugar de **/transparencia/**). Esto permite, por ejemplo, que un usuario sin privilegios ingrese a la ruta de "**Gestión de Usuarios**", edite su propio perfil y se auto otorgue permisos de Administrador General, logrando una escalada de privilegios total que compromete la integridad del sistema.
- **Recomendación:** Implementar validación de roles de forma estricta en cada petición (endpoint) dentro del servidor (**Back-end**), asegurando que cualquier intento de acceso o modificación desde una URL no autorizada sea rechazado con un error HTTP 403 (Prohibido).

3.2. Principio de Menor Privilegio

- **Observación:** Al revisar la tabla de usuarios registrados, se detectó que la totalidad de las cuentas de los funcionarios municipales creadas fueron configuradas con el nivel_acceso = 10 (Administrador General), independientemente de sus funciones reales en la municipalidad.
- **Impacto:** Esta práctica debilita la lógica de control de acceso y expone al sistema a modificaciones destructivas irreversibles (sean accidentales o malintencionadas). Adicionalmente, expone innecesariamente a los funcionarios a riesgos de responsabilidad administrativa, al otorgarles acceso a funciones y módulos que no corresponden a su cargo. Sus usuarios quedan vinculados a posibles errores de auditoría de los cuales no deberían ser partícipes.
- **Recomendación:** Revisar y ajustar los privilegios otorgados al personal, aplicando estrictamente el **Principio de Menor Privilegio** (cada usuario solo debe recibir permisos acorde a sus funciones), permitiendo así una matriz de permisos segura y segmentada.

3.3. Configuración Permisiva de CORS

- **Observación:** El servidor **Back-end** está configurado globalmente para aceptar peticiones provenientes de cualquier origen web mediante la directiva **Access-Control-Allow-Origin: '*'.**
- **Impacto:** Esta configuración anula las protecciones de la política de mismo origen (**SOP**) del navegador. Al permitir que cualquier dominio externo interactúe con la API, el sistema queda expuesto a ser consumido desde clientes no autorizados o sitios web falsificados (**Phishing**). Dado que el sistema utiliza tokens simples en lugar de mecanismos de sesión robustos, un atacante podría alojar un clon de la interfaz de la municipalidad; este clon funcionaría sin problemas conectado a la API real, facilitando la captura de credenciales y tokens de sesión de los funcionarios sin que el servidor bloquee la conexión.
- **Recomendación:** Restringir la directiva CORS implementando una "lista blanca" (**whitelist**). El servidor solo debe aceptar peticiones que provengan exclusivamente al dominio o IP oficial donde se encuentra desplegado el cliente (**Front-end**) de la municipalidad.

3.4. Existencia de Rutas API sin uso (Shadow APIs)

- **Observación:** El código del servidor mantiene activos múltiples **endpoints** duplicados para la misma acción (ej. **/auth/login** y **/auth/login2**). La ruta antigua carece de validaciones modernas como el registro de auditoría o la verificación de roles.
- **Impacto:** Amplia involuntariamente la superficie de ataque. Si se parchea una vulnerabilidad en la ruta oficial, los atacantes podrán seguir vulnerando el sistema a través de la ruta antigua olvidada que sigue procesando datos.

- **Recomendación:** Eliminar todo el código muerto y consolidar las funciones en rutas únicas.

3.5. Asignación Insegura por Defecto (Violación de "Fail-Safe Defaults")

- **Observación:** El código Front-end (específicamente en **layout.js** y el **index.html** del login) utiliza una lógica condicional altamente insegura para el enrutamiento. Si un usuario inicia sesión y, por algún error o falta de configuración, no posee un rol asignado en la base de datos, el sistema le asigna estáticamente por defecto el rol de admin_general para determinar a qué panel redirigirlo.
- **Impacto:** En lugar de denegar el paso ante la falta de credenciales específicas, o asignar los permisos más bajos, el sistema otorga el nivel de privilegio máximo. Esto genera un riesgo crítico de escalada de privilegios en el cliente, permitiendo a usuarios no configurados aterrizar directamente en paneles administrativos, dependiendo exclusivamente de las validaciones del Back-end (las cuales, como se comprobó, no validan permisos al momento de ejecutar acciones) para detenerlos.
- **Recomendación:** Modificar la lógica de enrutamiento y autenticación para aplicar denegación por defecto. Si un usuario no posee un rol o división válida, el sistema debe bloquear el redireccionamiento, mostrar un mensaje de error de "**Usuario sin perfil configurado"** y abortar la carga de cualquier interfaz administrativa.

3.6. Falta de Auditoría en la Cadena de Suministro (Dependencias de Terceros)

- **Observación:** El Back-end de la aplicación depende de múltiples librerías de terceros (ej. **Flask**, **psycopg2**, **bcrypt**, **Pillow**). Durante la revisión, no se evidenció la existencia de un control estricto de versiones (fijación de versiones) ni la integración de herramientas automatizadas para el escaneo de vulnerabilidades de estas dependencias.
- **Impacto:** Utilizar componentes desactualizados o vulnerables es uno de los riesgos más críticos según **OWASP**. Si una de las librerías base (como el framework web o el procesador de imágenes) contiene una falla de seguridad pública (**CVE**), los atacantes pueden vulnerar el servidor (ej. **Ejecución Remota de Código**) sin importar qué tan seguro sea el código escrito por los desarrolladores.
- **Recomendación:** Generar un archivo de dependencias estricto (ej. **requirements.txt** con versiones fijadas). Además, implementar herramientas de análisis de composición de software (SCA) como **pip-audit** y/o **Dependabot** en el repositorio, para detectar vulnerabilidades en librerías y gestionar sus respectivos parches de seguridad.

4. Resultados de Análisis Estático de Código (SAST)

4.1. Riesgo de Inyección SQL en el Back-end

- **Observación:** En funciones críticas como la creación y edición de proyectos (**create_proyecto** y **update_proyecto**), el servidor construye dinámicamente partes de las consultas SQL (como nombres de columnas) utilizando directamente la información enviada desde el cliente (**request.get_json()**). Si bien los valores son enviados mediante parámetros seguros (**%s**), no existe validación sobre qué campos pueden ser incluidos en la consulta, confiando completamente en la estructura de los datos entrantes.
- **Impacto:** Esta práctica introduce un riesgo de inyección SQL a nivel estructural. Un atacante podría manipular los campos enviados para alterar la estructura de la consulta, insertar columnas no previstas o generar errores que afecten la integridad y disponibilidad del sistema. En escenarios donde existan campos sensibles en la tabla, esto podría permitir modificaciones no autorizadas o escalamiento de privilegios.
- **Recomendación:** Definir explícitamente una lista de campos permitidos (**whitelist**) y filtrar los datos entrantes en base a esta. Evitar la construcción dinámica de elementos estructurales del SQL (como nombres de columnas) a partir de entrada del usuario. Para mayor robustez, considerar el uso de un ORM que gestione automáticamente la generación segura de consultas.

4.2. Riesgo de Cross-Site Scripting (XSS) en el Front-end

- **Observación:** Se identificó el uso generalizado de la propiedad **.innerHTML** en los archivos JavaScript (ej. **layout.js**) para renderizar datos en la interfaz.
- **Impacto:** Si el dato no está controlado/sanitizado, este diseño potencialmente permite ejecución de script del lado del cliente (**XSS**), con riesgo de robo de sesión.
- **Recomendación:** Sustituir por **.textContent/creación de nodos** y/o sanitización robusta antes del renderizado (p.ej., DOMPurify), especialmente en contenidos provenientes de usuario o servidor.

4.3. Integridad de Dependencias (Ataque de Cadena de Suministro)

- **Observación:** La totalidad de las librerías externas cargadas desde **CDNs** públicos carecen del atributo de seguridad **integrity** (SRI).
- **Impacto:** Si uno de estos servidores externos es vulnerado, un código malicioso podría descargarse y ejecutarse automáticamente en el equipo del usuario (Ataque de Cadena de Suministro).
- **Recomendación:** Fijar las versiones exactas de las librerías externas. Posteriormente, implementar Subresource Integrity (SRI) agregando obligatoriamente los atributos integrity (con la firma criptográfica base64) y crossorigin="anonymous" en todos los enlaces <script> y <link> externos.

5. Estabilidad de Infraestructura y Gobernanza de Datos

5.1. Interrupciones por mal manejo del Connection Pool (Errores 500)

- **Observación:** Se han registrado fallos en el primer intento de inicio de sesión (HTTP 500). El análisis de logs reveló el error **SSL connection has been closed unexpectedly**.
- **Causa Raíz:** El proveedor de base de datos (**NeonDB**) cierra conexiones inactivas para ahorrar recursos. El Back-end ignora esto y entrega conexiones "**muertas**" a la aplicación al no realizar comprobaciones previas.
- **Impacto:** Interrupción intermitente de la disponibilidad del servicio. El sistema rechaza peticiones válidas arrojando errores críticos (HTTP 500) de manera impredecible para el usuario final. Esto no solo entorpece la operación diaria de sus usuarios, sino que satura los registros (logs) del servidor con errores de conexión, dificultando el monitoreo y diagnóstico de fallas reales en la infraestructura.
- **Recomendación:** Implementar un mecanismo de "Comprobación de Salud" (*Pre-ping*) en la función **get_db_connection()** para verificar y desechar conexiones cerradas silenciosamente antes de procesar una petición.

5.2. Privacidad de Datos y Seguridad Crítica en IA (Chatbot)

- **Observación:** El chatbot envía el contexto de los proyectos (montos, aprobaciones y documentos internos) directamente desde el navegador hacia un proveedor internacional de IA (open.bigmodel.cn, infraestructura perteneciente a la empresa tecnológica china Zhipu AI).
- **Impacto Legal y de Seguridad:** El envío de borradores financieros, datos estratégicos y documentos de licitaciones a jurisdicciones extranjeras sin un proceso previo de anonimización o contratos Enterprise de confidencialidad, plantea un riesgo potencial de cumplimiento normativo y protección de datos respecto a la gobernanza y privacidad de la información municipal. Adicionalmente, la clave de la API (API_KEY) reside visible en el código cliente, esto permite que cualquier usuario con acceso al sistema pueda extraerla y hacer uso del servicio a discreción.
- **Recomendación:** Deshabilitar la comunicación directa desde el cliente. El módulo de IA debe enrutarse obligatoriamente a través del servidor Back-end (Patrón Proxy). El Back-end debe ser el único encargado de poseer las credenciales de forma segura, comunicarse con el servicio externo, y mantener un registro (log) de auditoría sobre qué información se exporta.

5.3. Filtración de Credenciales y Código Sensible Expuesto

- **Observación:** Se detectaron scripts de depuración (ej. **debug_users.py**), y archivos con variables de entorno (**.env_neon**) versionados en el repositorio

del código, los cuales contienen explícitamente y en texto plano la cadena de conexión completa de la base de datos.

- **Impacto:** Compromiso total de la base de datos. Cualquier individuo con acceso a estos archivos puede conectarse remotamente al servidor para extraer o manipular información. En el peor de los casos, permite realizar cambios destructivos en la estructura de la base de datos si el usuario de conexión expuesto posee privilegios excesivos de administración (violando el Principio de Menor Privilegio a nivel de infraestructura).
- **Recomendación:** Rotar de inmediato las contraseñas y eliminar permanentemente estos archivos del historial del repositorio. Asimismo, auditar y restringir los permisos del usuario de la base de datos utilizado por la aplicación para que solo posea privilegios de lectura/escritura sobre las tablas necesarias.

5.4. Credenciales por Defecto y Datos Semilla Inseguros

- **Observación:** Se identificaron archivos de inicialización de base de datos (**database/login.sql**) versionados en el repositorio que contienen la inserción de cuentas de administrador genéricas (ej. admin_general) preconfiguradas con contraseñas débiles y predecibles (ej. "123456") y sus respectivos hashes.
- **Impacto:** Mantener credenciales por defecto conocidas ("Default Credentials") es una de las vulnerabilidades más críticas según el estándar **OWASP**. Si la plataforma es desplegada a producción conservando estos registros de prueba, un atacante potencialmente puede obtener acceso garantizado con privilegios máximos de forma inmediata.
- **Recomendación de Producción:** Previo a la salida a producción, se debe realizar una purga de todos los usuarios de prueba. **Cabe destacar que no se solicita la eliminación de los scripts de inicialización (.sql) del repositorio, sino su estricta sanitización:** ninguna credencial base, contraseña genérica o hash real debe quedar escrita en el código fuente. Finalmente, el proceso de creación de usuarios reales debe forzar un cambio de contraseña obligatorio durante el primer inicio de sesión.

5.5. Operaciones con Transaccionalidad Insuficiente en Flujos Críticos

- **Observación:** Se detectó un patrón de diseño frágil en la gestión de operaciones lógicas que deberían ser atómicas (todo o nada). El sistema tiende a fragmentar procesos críticos. Por ejemplo, al crear o editar un usuario, el **Front-end** realiza llamadas API separadas (una para datos básicos y otra para asignar el rol). Asimismo, en el **Back-end**, el módulo de auditoría gestiona sus propios guardados independientes de la transacción principal.
- **Impacto:** Alto riesgo de inconsistencia de datos (corrupción estructural). Si ocurre una micro desconexión, caída de red o error de validación en medio de estas operaciones fragmentadas, la base de datos queda en un estado parcial. En el caso de los usuarios, esto genera cuentas "huérfanas" que rompen de

forma crítica el inicio de sesión. En el caso de la auditoría, puede generar registros de acciones exitosas (ej. "**Proyecto creado**") cuando en realidad la operación principal falló y fue revertida.

- **Recomendación:** Refactorizar la arquitectura para garantizar la integridad transaccional (estándar **ACID**). El Front-end debe enviar un único paquete de datos (**payload**) por cada acción lógica. A su vez, el Back-end debe procesar todas las inserciones y actualizaciones relacionadas (incluyendo la auditoría) dentro de una única transacción SQL atómica, asegurando que, ante cualquier error, se revierta la operación completa (**rollback**).

5.6. No se implementa JWT y Sesiones Volátiles

- **Observación:** Aunque la documentación técnica indica el uso del estándar *JSON Web Tokens* (**JWT**), en la realidad el código revela la creación de identificadores temporales simples (cadenas de texto aleatorias) almacenados únicamente en la memoria RAM del proceso de Python (**active_sessions** = {}).
- **Impacto:** Esta arquitectura es frágil. Cualquier mantenimiento o reinicio automático del servidor en la nube (ej. **Railway**) borrará la memoria RAM, expulsando abruptamente a todos los funcionarios del sistema y causando la pérdida de cualquier trabajo no guardado.
- **Recomendación:** Migrar a un sistema de JWT criptográfico real (Stateless) o respaldar las sesiones activas en la base de datos.

6. Higiene del Repositorio y Deuda Técnica

6.1. Falta de Limpieza y Archivos Residuales

- **Observación:** El repositorio contiene múltiples archivos residuales, de pruebas o vacíos, que no cumplen función operativa en producción (ej. **nginx.config** vacío, **check_db.py**, **neon.py**, **preguntas.txt** sin funcionalidad clara).
- **Impacto:** Genera "ruido" arquitectónico en el repositorio, lo cual dificulta la comprensión de la arquitectura real para los futuros desarrolladores que asuman el soporte y mantenimiento del software.
- **Recomendación:** Realizar una limpieza estructural del repositorio, eliminando todo archivo de desarrollo obsoleto y scripts de pruebas antes del paso definitivo a producción.

6.2. Enlaces Rígidos (Hardcoding de IP) en el Front-end

- **Observación:** La dirección IP del servidor está escrita de forma estática directamente en el código de archivos HTML y JavaScript (ej. **index.html**, **api.js**) en distintos sitios (se encontraron más de 25 coincidencias).

- **Impacto:** Si en algún momento se migra el servicio a otra infraestructura (como a un servidor local de la municipalidad) o por algún motivo cambia la IP/dominio actual, el sistema entero dejará de funcionar. Esta práctica es incompatible con despliegues modernos (ej. **Docker**), ya que requeriría una intervención manual exhaustiva en el código para volver a operar.
- **Recomendación:** Modificar el Front-end para utilizar rutas relativas o centralizar la configuración de red en un único archivo global (ej. **config.js** o **variables de entorno**), permitiendo que las transiciones de infraestructura sean más limpias.

6.3. Interfaces Duplicadas (Shadow UI)

- **Observación:** Se detectó la existencia de interfaces de usuario duplicadas y abandonadas en el código fuente (ej. una vista alternativa de Gestión de Usuarios accesible vía URL directa). Esta interfaz no es completamente funcional y presenta una lógica desalineada con el Back-end actual.
- **Impacto:** Mantener código "muerto" o interfaces ocultas operativas genera confusión arquitectónica, ensucia el repositorio y, lo más grave, amplía innecesariamente la superficie de ataque del sistema al dejar expuestas rutas que ya no reciben mantenimiento ni pruebas de seguridad.
- **Recomendación:** Eliminar permanentemente del repositorio todas las vistas (HTML), scripts y rutas que hayan sido desechadas y no formen parte del flujo oficial del producto final.

6.4. Bug de Sobrescritura de Datos (Pérdida de Contraseñas)

- **Observación:** Al editar el perfil de un usuario desde la interfaz oficial, el Front-end envía el campo de la contraseña en blanco. Paralelamente, el Back-end (**update_user**) carece de validaciones de integridad y procesa este campo vacío sin discriminación.
- **Impacto:** Al no usar programación defensiva, el servidor encripta una cadena vacía (nada) y sobrescribe la contraseña real por una vacía en la base de datos. Esto provoca que los administradores destruyan involuntariamente cualquier acceso al sistema al intentar realizar un cambio menor, como editar un rol, requiriendo intervención manual en la base de datos para restaurar la contraseña o desde otra cuenta de usuario.
- **Recomendación:** Implementar validaciones estrictas en el Back-end, asegurando que los campos críticos (especialmente las contraseñas) sean ignorados durante una operación de actualización si se reciben con valores nulos o vacíos.

6.5. Manejo Deficiente de Excepciones en el Cliente (Enmascaramiento de Errores)

- **Observación:** Si bien es una buena práctica de seguridad usar mensajes genéricos ante fallos de autenticación, el Front-end no discrimina los códigos

de estado HTTP del servidor. Captura errores de infraestructura (ej. HTTP 500 por desconexión de la base de datos o caídas de red) y los enmascara bajo el mismo mensaje de “Credenciales inválidas”.

- **Impacto:** El sistema oculta activamente las caídas del servidor al usuario final. Esto no solo genera frustración al hacerle creer al funcionario que olvidó su clave, sino que provoca falsos reportes a soporte técnico, ocultando los verdaderos problemas de infraestructura y dificultando el diagnóstico en producción.
- **Recomendación:** Implementar un interceptor de errores adecuado en el Front-end. Los errores de validación (401/404) deben mantener su ambigüedad por seguridad, pero los errores críticos de servidor (500) o caídas de conexión deben notificar claramente al usuario que el sistema o la red presentan problemas temporales.

6.6. Diccionario de Rutas Incompleto y Bucles de Redirección

- **Observación:** El sistema de enrutamiento del Front-end (diccionarioRutas) se encuentra incompleto. Únicamente mapea las vistas para los roles de administración superior (Niveles 10, 11 y 12), dejando sin interfaz asignada a los roles inferiores. Adicionalmente, la lógica de contingencia (*fallback*) para roles no mapeados utiliza rutas relativas incorrectas.
- **Impacto:** Si un funcionario con un rol no mapeado (ej. Rol **"Transparencia"**) intenta iniciar sesión en el entorno de producción, el sistema no le restringe el acceso limpiamente. En su lugar, entra en un bucle de redirección infinita (**/frontend/frontend/frontend/...**) bloqueando el navegador del usuario y degradando por completo la experiencia de uso.
- **Recomendación:** Resolver el enrutamiento mediante tres acciones clave: **a)** Completar el diccionario para cubrir la totalidad de los roles de la base de datos; **b)** Implementar una vista dedicada de “Acceso Denegado” (HTTP 403) para derivar a los usuarios sin panel de control asignado; y **c)** Utilizar rutas absolutas para las redirecciones automáticas en JavaScript, previniendo así las concatenaciones en bucle originadas por subdirectorios.

7. Inconsistencias en la Documentación Técnica

Para asegurar la mantenibilidad a futuro por parte del equipo municipal, el código debe coincidir fielmente con su documentación oficial. Actualmente, se identificó una divergencia significativa entre lo declarado y lo entregado:

- **Implementación no Alineada con Estándar Declarado:** La documentación indica el uso de "JWT Auth" (JSON Web Tokens) para la seguridad. Sin embargo, el código implementa identificadores temporales simples almacenados en estructuras volátiles de memoria (RAM).

- **Fuga de Información (Back-end):** El servidor utiliza capturas genéricas (**except Exception**) que devuelven rastros internos crudos al cliente (**str(e)**), exponiendo detalles internos sobre la estructura.
- **Stack Tecnológico Divergente:** Los lineamientos originales indican el uso del framework FastAPI para el desarrollo del Back-end, pero la revisión del código fuente confirma que el sistema está construido íntegramente sobre Flask (ej. en **app21.py**).
- **Capacidades Declaradas no Alineadas con el Comportamiento Observado:** El documento **propuesta.md** ofrece características como "API de Alto Rendimiento" y "Gestión Inteligente de Errores". Sin embargo, se observó errores recurrentes por desconexión de base de datos (HTTP 500), la ausencia de mecanismos de recuperación automática ante fallos de conexión y una arquitectura con escalabilidad limitada debido al almacenamiento de sesiones en RAM.
- **Recomendación:** Alinear la documentación técnica (**README_TECH.md** y **TECHNICAL_DOC.md**) y comercial (**propuesta.md**) con la arquitectura y capacidades efectivamente implementadas, para que refleje con exactitud la arquitectura, librerías, capacidades reales y lógicas de seguridad que están operando en el producto entregado a la municipalidad.

8. Problemas de Usabilidad en Dispositivos Móviles (Diseño Responsivo)

Si bien el uso principal del sistema puede estar pensado para computadores de escritorio, la plataforma presenta fallas críticas de interfaz al acceder desde teléfonos móviles:

- **Bloqueo de Navegación:** El menú lateral (Sidebar) se oculta automáticamente en pantallas pequeñas (comportamiento esperado), pero no ofrece un botón de despliegue ("hamburguesa"), atrapando al usuario móvil en la vista inicial e impidiéndole navegar por los módulos.
- **Desbordamiento de Tablas:** Las tablas de datos de los distintos módulos carecen de contenedores responsivos. Esto provoca que la información más ancha se desborde, rompiendo los márgenes de la pantalla, desconfigurando la estructura visual y haciendo difícil leer o gestionar los registros correctamente desde un dispositivo móvil.

Recomendación (Opcional): Queda a criterio de SECPLAC definir el alcance operativo del sistema. Se debe decidir si se exigirá la corrección del Front-end (Diseño responsivo) para permitir la administración remotamente, o si se restringirá el uso del panel web exclusivamente a equipos de escritorio de oficina.

9. Alcance del Proyecto y Módulos no Solicitados

- **Observación:** Se detectó la presencia de módulos funcionales complejos en el Back-end (ej. **Portal Vecino**, **API Móvil**, roles de "**Vecino**" y "**Fiscalizador**") y apartados visuales sin funcionalidad real en el Front-end (placeholders de otros departamentos), los cuales exceden el enfoque original de gestión interna de proyectos planteado para SECPLAC. Además, la interfaz web de la aplicación "Vecinos" se renderiza estáticamente dentro del marco de un teléfono móvil simulado (incluso al visualizarse desde un teléfono real), confirmando visualmente que se trata de una maqueta rápida (Mockup/Prueba de Concepto) y no de una vista de producción.
- **Impacto:** En ingeniería de software, el código no utilizado o a medio terminar no es un beneficio, sino una responsabilidad heredada (Deuda Técnica). Mantener estas características amplía innecesariamente la superficie de ataque del sistema frente a vulnerabilidades y obliga a la municipalidad a mantener, actualizar y auditar cientos de líneas de código que no aportan valor directo a la operabilidad actual del departamento.
- **Recomendación:** Se sugiere evaluar la pertinencia estratégica de estas funciones. Si no existe un plan de implementación real o respaldo institucional para el uso ciudadano/externo de la plataforma, se recomienda solicitar la amputación y limpieza de todo este código "excedente", asegurando la recepción de un sistema limpio, seguro y acotado estrictamente a lo solicitado.

10. Conclusión General

El desarrollo evaluado cuenta con la funcionalidad base solicitada, pero su código fuente revela que fue construido bajo estándares propios de un prototipo rápido (***Prueba de concepto***) y no de un software a nivel de producción. La presencia de vulnerabilidades críticas, la falta de transaccionalidad, el uso de código duplicado, los bugs lógicos destructivos (como la sobrescritura de contraseñas) y la inclusión de módulos extra no solicitados (y sin terminar) que desvían el alcance original, **hacen técnica y operativamente inviable su paso a producción sin antes someterse a un ciclo profundo de refactorización.**

Se recomienda requerir la subsanación integral de los hallazgos de Seguridad y Arquitectura (Secciones 3, 4 y 5) y la corrección inmediata de la Deuda Técnica (Sección 6) como condición para la puesta en marcha del sistema. Adicionalmente, es imperativo que la documentación técnica y comercial refleje la realidad del sistema (Sección 7), y se recomienda evaluar estratégicamente el alcance operativo final respecto a la usabilidad móvil (Sección 8) y la retención o eliminación de los módulos excedentes (Sección 9).